mra.py 最近一次提交变更说明

2026-03-30

1 文档范围

本文档说明 `mra.py` 在最近一次涉及该文件的提交中发生了哪些变化。比较基准为：

- 当前提交：`3ac17056cbf19bdc0188b473956c9b438af3ceb0`
- 提交信息：`IMPORTANT: optimize mra.py`
- 提交时间：2026-03-30 03:55:08 +0800
- 对比对象：该提交的父提交版本

另外，当前工作区相对 `HEAD` 没有新的未提交改动。因此，本文档描述的是“最近一次**已提交**变更”，而不是“当前未提交 diff”。

2 变更规模与总体结论

根据 `git diff --numstat HEAD^ HEAD -- mra.py` 的结果，本次提交对 `mra.py` 的改动规模为：

- 新增 360 行；
- 删除 97 行。

从整体上看，这次提交**没有大改 AGF-ADNet 主体网络结构**，重点变化集中在训练与检测流程上。脚本的角色从“单模型重构误差检测脚本”，调整为更偏向“可复现实验 + 多种子集成 + 持续故障检测”的完整 pipeline。

3 高层变化概览

模块	旧版本	新版本
运行环境与可复现性	没有统一随机种子入口；未显式设置 Matplotlib 缓存目录	新增 `seed_everything()`、EPS 常量，以及 `MPLCONFIGDIR=/tmp/matplotlib`

模块	旧版本	新版本
标准化逻辑	DatasetBuilder 内部直接持有 StandardScaler，以 0 填充 NaN 后缩放	DatasetBuilder 改为支持 NaN-aware 均值/方差；但 train() 主路径又显式回退到 0 填充 + StandardScaler 基线
训练方式	单个 AGF_ADNet 训练 3 个 epoch，直接取最佳 state	抽出 train_single_agf_model()，按多个 seed 训练，并支持 checkpoint 读写
检测策略	训练集分数阈值 = 均值 + 2 倍标准差；测试集逐点判异常	引入 EWMA 平滑、最短持续长度约束，面向持续故障检测
评估输出	只输出一套点式分类指标	同时输出原始逐点评估与持续故障评估两套指标
可视化	只画单条异常分数曲线	可叠加原始分数与平滑分数，图标题与坐标含义同步更新

4 逐项变更说明

4.1 运行时与基础设施改造

文件头部新增了 copy、os、random 的整理导入，并补充了：

- os.environ.setdefault("MPLCONFIGDIR", "/tmp/matplotlib")；
- EPS = 1e-6；
- seed_everything(seed=40)。

这部分改动的直接作用有两个：

- 让脚本在受限环境里使用 Matplotlib 时更稳定，减少缓存目录权限问题；
- 让 NumPy、PyTorch、CUDA 相关随机行为尽量固定，提升多次实验的一致性。

4.2 数据标准化逻辑发生了“探索后回退”的变化

旧版本中,DatasetBuilder 的 fit_scaler() 与 transform() 都依赖内部保存的 StandardScaler。处理方式很直接：先把 NaN 位置填成 0，再做标准化。

新版本里，DatasetBuilder 被改造成了另一套接口：

- 不再保存 self.scaler；
- 改为保存 feature_mean 和 feature_std；
- 只基于观测值计算均值与标准差；
- 缺失位置在缩放后被重新置为 0。

这说明作者曾尝试把预处理改成“对缺失值更友好”的 NaN-aware 标准化。

但是，`train()` 主流程随后又显式写回了另一条路径：

- 先对训练集和测试集做 `np.nan_to_num(..., nan=0.0)`；
- 再重新实例化一个局部 `StandardScaler()`；
- 并在注释中明确说明：保留原本的 0 填充标准化，因为它在该数据集上效果更好，还能保留缺失模式线索。

因此，这次提交在标准化部分的真实结论不是“彻底切换到了 NaN-aware 标准化”，而是：

> 代码里增加了一套 NaN-aware 预处理实现，但当前主训练路径最终仍然采用原始的 0 填充 + `StandardScaler` 方案。

4.3 新增了一批检测辅助函数

本次提交在 `anomaly_scores()` 前后新增了多组辅助函数，作用可以分成两类。

第一类：面向去噪/代理异常验证的实验性函数

- `build_type_index()`
- `build_denoising_masks()`
- `create_pseudo_anomalies()`
- `evaluate_proxy_detection()`

这些函数的意图比较明显：

- 构造额外遮挡掩码；
- 人工生成伪异常窗口；
- 用“正常窗口 vs 伪异常窗口”的可分性来做代理验证。

不过从当前代码调用关系看，这些函数**尚未接入** `train()` 主流程。也就是说，它们更像是为后续实验预留的能力，而不是已经生效的正式训练逻辑。

第二类：面向持续故障检测的正式函数

- `ewma_smooth()`
- `enforce_min_run()`
- `persistent_fault_detection()`

这三者已经被主流程实际调用，构成了新的检测后处理逻辑：

1. 先对训练分数和测试分数做 EWMA 平滑；
2. 再根据平滑后的训练分数计算阈值；
3. 最后把连续长度不足的短暂告警过滤掉，只保留持续时间足够长的异常段。

这意味着检测目标从“单点是否越阈值”，转向了“是否出现了足够持续的故障区间”。

4.4 训练逻辑从单模型改成多种子集成

旧版本的训练流程比较直接：

- 建一个 `AGF_ADNet`；
- 用 Adam 训练 3 个 epoch；
- 使用 `ReduceLROnPlateau` 调整学习率；
- 保存最佳 state；
- 用单模型在训练集上估阈值，在测试集上评估。

新版本把这部分重构成了两层逻辑。

单模型训练层 新增 `train_single_agf_model()`，负责：

- 设定随机种子；
- 构建模型；
- 可选地从 checkpoint 直接加载；
- 否则执行训练、梯度裁剪、最佳参数保存；
- 训练完成后把模型写回 checkpoint。

外层集成层 `train()` 里新增了：

- `ENSEMBLE_SEEDS = (40, 41, 42)`；
- `CKPT_DIR = "./agf_adnet_checkpoints"`；
- 对每个 seed 分别训练/加载一个模型；
- 最后对多模型的训练分数和测试分数做平均。

因此，当前脚本的核心检测器不再是“一个 AGF-ADNet”，而是“多个种子训练出来的 AGF-ADNet 集成均值分数”。

4.5 阈值与评估目标被重新定义

旧版本中，核心判定规则是：

$$\text{threshold} = \mu_{\text{train}} + 2\sigma_{\text{train}} \tag{1}$$

然后直接对测试集分数逐点二值化。

新版本把评估拆成了两层：

- 原始逐点评估：先计算原始分数，再用 `mean + 1std` 得到一个原始阈值，并输出一套 raw metrics；

- 持续故障评估：对分数做 EWMA 平滑，并使用

$$\text{threshold} = \mu_{\text{smoothed train}} + 1.5\sigma_{\text{smoothed train}} \tag{2}$$

再叠加最短持续长度约束，输出 persistent metrics。

默认超参数也因此更偏“持续事件检测”：

- `EWMA_ALPHA = 0.02`
- `THRESHOLD_STD = 1.5`
- `MIN_RUN = 150`

这部分是本次提交最重要的语义变化：**脚本不再只是做逐点异常检测，而是在显式追求持续故障的稳定识别**。

4.6 结果输出与绘图方式同步调整

`plot_results()` 也随之被改写：

- 新增 `raw_scores=None` 参数；
- 可以在图中同时绘制原始异常分数和平滑异常分数；
- 纵轴名称从“重构误差”改成“异常分数”；
- 图标题从“AGF-ADNet 异常检测”改成“AGF-ADNet 持续故障检测”；
- 保存图片后不再 `plt.show()`，而是 `plt.close()`。

这些改动和前面的训练/评估调整是完全一致的：可视化的解释对象也从单点误差转向了平滑后的持续异常趋势。

5 当前代码状态中的几个注意点

- `fit_scaler()` 和 `transform()` 仍保留在 `DatasetBuilder` 中，但当前 `train()` 主流程没有调用它们。
- `build_type_index()`、`build_denoising_masks()`、`create_pseudo_anomalies()`、`evaluate_proxy_detectio` 当前也没有接入正式流程。
- `train()` 里仍然保留了一个 `train_loader` 局部变量，但后续实际上没有再使用它；这更像是重构后留下的残余代码。
- 主体网络模块（图学习、GCN、TCN、频域分支、Transformer 头等）基本保持不变，所以本次提交的重点是 pipeline 优化，而不是 backbone 改写。

6 一句话总结

如果用一句话概括本次提交，那么最准确的表述是：

> `mra.py` 从“单模型、单阈值、逐点异常判定”的脚本，升级成了“可复现、多种子集成、带平滑与持续长度约束的持续故障检测 pipeline”；同时保留了一些尚未正式接入主流程的实验性函数与预处理分支。